

Halle Nkosi

ST10471832

PROG 6212 Part 1

Section A

Race Day System – Entity Diagram

Organisers
PK OrganiserID (int)
FullName (varchar)
Email (varchar, unique)
PasswordHash (varchar)
Phone (varchar)
CreatedAt (datetime)

Events
PK EventID (int)
FK OrganiserID (int)
EventName (varchar)
EventDate (date)
Location (varchar)
Description (varchar)
CreatedAt (datetime)

Categories
PK CategoryID (int)
FK EventID (int)
CategoryName (varchar)
DistanceKm (decimal)
MaxParticipants (int)
Price (decimal)

Enrolments
PK EnrolmentID (int)
FK CategoryID (int)
FK ParticipantID (int)
EnrolmentDate (datetime)
Status (varchar)

Participants
PK ParticipantID (int)
FullName (varchar)
Email (varchar, unique)
PasswordHash (varchar)
Phone (varchar)
DateOfBirth (date)
CreatedAt (datetime)

Results
PK ResultID (int)
FK EnrolmentID (int)
FinishTime (time)
Position (int)
Status (varchar)

creates

1

M

has

1

M

receives

M

makes

produces

M

1

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
POST	/api/auth/register/organizer	Registers a new organiser account.	None	{ fullName, email, password, phone }	201 Created – new organiser record (no password). 409 Conflict – email already registered. 400 Bad Request – missing/invalid fields.
POST	/api/auth/register/participant	Registers a new participant account.	None	{ fullName, email, password, phone, dateOfBirth }	201 Created – new participant record (no password). 409 Conflict – email already registered. 400 Bad

					Request – missing/invalid fields.
POST	/api/auth/login	Authenticate a user (organiser or participant) and returns a token.	None	{ email, password }	200 OK – { token, role, userId }. 401 Unauthorized – invalid credentials.

User Profile

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
GET	/api/users/me	Returns the logged-in user's own profile.	Any	None	200 OK – profile object. 401 Unauthorized – no/invalid token.
PUT	/api/users/me	Updates the logged-in user's own profile details.	Any	{ fullName, phone, ...role-specific fields }	200 OK – updated profile. 400 Bad Request – invalid fields.

					401 Unauthorized.
--	--	--	--	--	----------------------

Events

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
GET	/api/events	Lists all upcoming events (public event catalogue).	None	None	200 OK – array of events.
GET	/api/events/{id}	Retrieves the details of a single event	None	None	200 OK – event object. 404 Not Found.
POST	/api/events	Creates a new event, owned by the logged-in organizer.	Organizer	{ eventName, eventDate, location, description }	201 Created – new event. 400 Bad Request – invalid fields.

					401 Unauthorized.
PUT	/api/events/{id}	Updates an event owned by the logged-in organizer.	Organizer	{ eventName, eventDate, location, description }	200 OK – updated event. 403 Forbidden – not the owner. 404 Not Found.
DELETE	/api/events/{id}	Deletes an event owned by the logged-in organizer.	Organizer	None	204 No Content. 403 Forbidden – not the owner. 404 Not Found.

Categories

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
GET	/api/events/{eventId}/categories	Lists all race categories for a given event.	None	None	200 OK – array of categories. 404 Not Found – event does not exist.
POST	/api/events/{eventId}/categories	Adds a new category to an event owned by the logged-in organizer.	Organizer	{ categoryName, distanceKm, maxParticipants, price }	201 Created – new category. 400 Bad Request. 403 Forbidden – not the event owner.

PUT	/api/categories/{id}	Updates a category belonging to the logged-in organizer's event.	Organizer	{ categoryName, distanceKm, maxParticipants, price }	200 OK – updated category. 403 Forbidden. 404 Not Found.
DELETE	/api/categories/{id}	Deletes a category belonging to the logged-in organizer's event.	Organizer	None	204 No Content. 403 Forbidden. 404 Not Found.

Event Enrolments

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
POST	/api/categories/{id}/enrol	Enrolls the logged-in participant	Participant	None	201 Created – enrolment record with

		into a race category.			status "Confirmed". 409 Conflict – already enrolled, or category full. 404 Not Found – category does not exist.
GET	/api/participants/me/enrolments	Lists the logged-in participant's own enrolments.	Participant		200 OK – array of enrolments
GET	/api/categories/{id}/enrolments	Lists everyone enrolled in a category (for the organizer	Organizer	None	200 OK – array of enrolled participants . 403 Forbidden –

		who owns it).			not the event owner
DELETE	/api/enrolments/{id}	Cancels the logged-in participant's own enrolment.	Participant	None	204 No Content. 403 Forbidden – not the owner of the enrolment. 404 Not Found.

Results

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
POST	/api/enrolments/{id}/result	Captures a finish time/position for an enrolment (organizer	Organizer	{ finishTime , position, status }	201 Created – new result. 403 Forbidden – not the

		marks results after the race).			event owner. 404 Not Found – enrolment does not exist.
GET	/api/categories/{id}/results	Public leaderboard of results for a category, ordered by position.	NONE	None	200 OK – array of results. 404 Not Found.
GET	/api/participants/me/results	Lists the logged-in participant's own race history/results	Participant	None	200 OK – array of results.

Section C: SQL Script

```
IF DB_ID('RaceDayDB') IS NOT NULL
```

```
BEGIN
```

```
    ALTER DATABASE RaceDayDB SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
```

```
    DROP DATABASE RaceDayDB;
```

```
END
```

```
GO
```

```
CREATE DATABASE RaceDayDB;
```

```
GO
```

```
USE RaceDayDB;
```

```
GO
```

TABLE: Organisers

```
CREATE TABLE Organisers (
```

```
    OrganiserID INT IDENTITY(1,1) PRIMARY KEY,
```

```
    FullName VARCHAR(100) NOT NULL,
```

```
    Email VARCHAR(150) NOT NULL UNIQUE,
```

```
    PasswordHash VARCHAR(255) NOT NULL,
```

```
    Phone VARCHAR(20) NULL,
```

```
    CreatedAt DATETIME NOT NULL DEFAULT GETDATE())
```

```
);
```

```
GO
```

TABLE: Participants

```
CREATE TABLE Participants (
```

```
    ParticipantID INT IDENTITY(1,1) PRIMARY KEY,
```

```
    FullName     VARCHAR(100) NOT NULL,
```

```
    Email        VARCHAR(150) NOT NULL UNIQUE,
```

```
    PasswordHash VARCHAR(255) NOT NULL,
```

```
    Phone        VARCHAR(20) NULL,
```

```
    DateOfBirth  DATE      NOT NULL,
```

```
    CreatedAt    DATETIME  NOT NULL DEFAULT GETDATE()
```

```
);
```

```
GO
```

TABLE: Events

CREATE TABLE Events (

EventID INT IDENTITY(1,1) PRIMARY KEY,

OrganiserID INT NOT NULL,

EventName VARCHAR(150) NOT NULL,

EventDate DATE NOT NULL,

Location VARCHAR(150) NOT NULL,

Description VARCHAR(500) NULL,

CreatedAt DATETIME NOT NULL DEFAULT GETDATE(),

CONSTRAINT FK_Events_Organisers FOREIGN KEY (OrganiserID)

REFERENCES Organisers(OrganiserID)

);

GO

TABLE: Categories


```
CREATE TABLE Categories (  
    CategoryID INT IDENTITY(1,1) PRIMARY KEY,  
    EventID INT NOT NULL,  
    CategoryName VARCHAR(100) NOT NULL,  
    DistanceKm DECIMAL(5,2) NOT NULL,  
    MaxParticipants INT NOT NULL DEFAULT 100,  
    Price DECIMAL(8,2) NOT NULL DEFAULT 0,  
    CONSTRAINT FK_Categories_Events FOREIGN KEY (EventID)  
        REFERENCES Events(EventID)  
);  
  
GO
```

TABLE: Enrolments (many-to-many between Participants and
Categories, resolved as its own entity)

```
CREATE TABLE Enrolments (  
    EnrolmentID INT IDENTITY(1,1) PRIMARY KEY,  
    CategoryID INT NOT NULL,  
    ParticipantID INT NOT NULL,  
    EnrolmentDate DATETIME NOT NULL DEFAULT GETDATE(),  
    Status VARCHAR(20) NOT NULL DEFAULT 'Confirmed',  
    CONSTRAINT FK_Enrolments_Categories FOREIGN KEY (CategoryID)  
        REFERENCES Categories(CategoryID),  
    CONSTRAINT FK_Enrolments_Participants FOREIGN KEY (ParticipantID)  
        REFERENCES Participants(ParticipantID),  
    CONSTRAINT UQ_Enrolments_Participant_Category UNIQUE (CategoryID, ParticipantID)  
);  
  
GO
```

TABLE: Results

```
CREATE TABLE Results (  
    ResultID    INT IDENTITY(1,1) PRIMARY KEY,  
    EnrolmentID INT    NOT NULL UNIQUE,  
    FinishTime  TIME    NULL,  
    Position    INT     NULL,  
    Status      VARCHAR(20) NOT NULL DEFAULT 'Finished',  
    CONSTRAINT FK_Results_Enrolments FOREIGN KEY (EnrolmentID)  
        REFERENCES Enrolments(EnrolmentID)  
);  
  
GO
```

SEED DATA

-- 2 Organisers

```
INSERT INTO Organisers (FullName, Email, PasswordHash, Phone) VALUES
```

```
('Thabo Nkosi', 'thabo.nkosi@raceday.co.za', 'hashed_pw_1', '0821234567'),  
( 'Sarah van Wyk', 'sarah.vanwyk@raceday.co.za', 'hashed_pw_2', '0837654321');
```

-- 2 Participants

```
INSERT INTO Participants (FullName, Email, PasswordHash, Phone, DateOfBirth) VALUES  
( 'Lindiwe Dube', 'lindiwe.dube@example.com', 'hashed_pw_3', '0791112222', '1996-04-12'),  
( 'James Botha', 'james.botha@example.com', 'hashed_pw_4', '0824445555', '1990-09-23');
```

-- 3 Events (owned by the 2 organisers)

```
INSERT INTO Events (OrganiserID, EventName, EventDate, Location, Description) VALUES  
(1, 'Benoni City Marathon', '2026-10-18', 'Benoni, Gauteng', 'Annual road race through  
Benoni CBD and surrounds.'),  
(1, 'Ekurhuleni Trail Run', '2026-11-08', 'Ekurhuleni, Gauteng', 'Off-road trail event across  
three distances.'),  
(2, 'Cape Coastal Fun Run', '2026-12-05', 'Cape Town, Western Cape', 'Family-friendly  
coastal run with a kids fun run.');
```

-- Categories for each event

```
INSERT INTO Categories (EventID, CategoryName, DistanceKm, MaxParticipants, Price)
VALUES
```

```
(1, '10km', 10.0, 500, 150.00),
```

```
(1, '21km', 21.1, 300, 250.00),
```

```
(2, '5km Trail', 5.0, 200, 120.00),
```

```
(2, '15km Trail', 15.0, 150, 200.00),
```

```
(3, '5km Fun Run', 5.0, 400, 100.00);
```

-- Sample enrolments (Participant 1 and 2 enter a few categories)

```
INSERT INTO Enrolments (CategoryID, ParticipantID, Status) VALUES
```

```
(1, 1, 'Confirmed'), -- Lindiwe -> Benoni 10km
```

```
(2, 2, 'Confirmed'), -- James -> Benoni 21km
```

```
(3, 1, 'Confirmed'), -- Lindiwe -> Ekurhuleni 5km Trail
```

(5, 2, 'Confirmed'); -- James -> Cape Coastal 5km Fun Run

-- Sample results for some enrolments

INSERT INTO Results (EnrolmentID, FinishTime, Position, Status) VALUES

(1, '00:52:14', 3, 'Finished'),

(2, '01:48:30', 12, 'Finished');

GO

Reference List

Chen, P.P., 1976. The entity-relationship model—toward a unified view of data. *ACM Transactions on Database Systems*, 1(1), pp.9–36.

Connolly, T. & Begg, C., 2015. *Database systems: a practical approach to design, implementation, and management*. 6th ed. Boston: Pearson.

Elmasri, R. & Navathe, S.B., 2016. *Fundamentals of database systems*. 7th ed. Boston: Pearson.

Fielding, R.T., 2000. *Architectural styles and the design of network-based software architectures*. PhD thesis. University of California, Irvine.

GitHub, 2024. *GitHub Actions documentation*. [online] Available at: <https://docs.github.com/en/actions> [Accessed 4 September 2026].

Microsoft, 2024. *CREATE TABLE (Transact-SQL)*. [online] Available at: <https://learn.microsoft.com/en-us/sql/t-sql/statements/create-table-transact-sql> [Accessed 4 September 2026].

Microsoft, 2024. *REST API guidelines*. [online] Available at: <https://github.com/microsoft/api-guidelines> [Accessed 4 September 2026].

Microsoft, 2024. *SQL Server Management Studio (SSMS) documentation*. [online] Available at: <https://learn.microsoft.com/en-us/sql/ssms/sql-server-management-studio-ssms> [Accessed 4 September 2026].

Microsoft, 2024. *Transact-SQL reference (Database Engine)*. [online] Available at: <https://learn.microsoft.com/en-us/sql/t-sql/language-reference> [Accessed 4 September 2026].

diagrams.net, 2024. *draw.io*. [online] Available at: <https://www.drawio.com/> [Accessed 4 September 2026].

